

Sección 10 Resiliencia de Datos y Alto Rendimiento

- Virtual Threads (Project Loom): Activación de hilos livianos de Java 21 para manejar concurrencia masiva con mínimo consumo de recursos.
- El problema de la dependencia del broker: Análisis de por qué un sistema que depende de RabbitMQ para guardar pedidos no puede considerarse confiable.
- Patrón Transactional Outbox: Implementación del mecanismo que desacopla el guardado del pedido de la entrega del mensaje, garantizando zero data loss.
- Message Relayer: Proceso scheduler que lee los mensajes pendientes de la base de datos y los entrega al broker cuando está disponible.
- Coreografía de eventos: Diferenciación entre OrderPlaced y OrderConfirmed para evitar bucles infinitos y garantizar el flujo correcto del saga.

** Un Message Relayer (relevador de mensajes) es un componente de software diseñado para mover mensajes, eventos o datos desde un sistema de almacenamiento temporal (como una base de datos o una cola local) hacia un sistema externo (como un bróker de mensajería como Kafka o RabbitMQ). Se utiliza principalmente para implementar el Patrón Outbox, asegurando que los datos se guarden en la base de datos y se envíen al mundo exterior de forma segura y sin pérdidas.

150. Virtual Threads: Gateway y Notification

¿Por qué ahora?

El Gateway como cuello de botella

Ya activamos los hilos virtuales en nuestros servicios de lógica, pero hoy vamos a aplicarlos **donde más importa**: el API Gateway.

El Problema

Al ser la puerta de entrada única, si el Gateway se queda sin hilos del sistema operativo, **todo el sistema parece caído**.



Al activar **Project Loom**, el Gateway dejará de usar hilos pesados y pasará a usar **hilos virtuales**



Permitiendo que **miles de usuarios entren simultáneamente** sin aumentar el consumo de memoria

También lo activaremos en **Notification Service**, para que el envío de mails (una tarea lenta y bloqueante) no detenga el procesamiento de otros mensajes.



¿Por qué Project Loom?

La arquitectura detrás del cambio

Platform Threads (El límite)

Un **Platform Thread** en Java es un envoltorio de un hilo del Sistema Operativo (OS Thread). Esto es costoso: cada hilo reserva mucha memoria y el cambio de contexto consume alta CPU.

Project Loom es el rediseño de esta arquitectura en OpenJDK para permitir planificación de tareas masivas sin depender directamente del OS.



El "Telar" de la JVM

Cómo funciona el mapeo M:N

- 1 **Abstracción:** En lugar del OS, ahora es la JVM la que gestiona y controla los hilos.
- 2 **Mapeo M:N:** El "Telar" monta miles de hilos virtuales sobre un grupo pequeño de hilos de plataforma (Carrier Threads).
- 3 **Desacoplamiento:** Si un hilo virtual se bloquea (I/O), la JVM lo desmonta automáticamente, liberando el hilo de plataforma.

Resultado: Logramos una densidad de concurrencia imposible antes en Java, optimizando memoria y CPU al máximo.

→ En api-gateway y notification-service.yml:

```
Project
└─ microservices-ecommerce D:\new_projects\
    └─ .idea
    └─ api-gateway
    └─ config-data
        └─ api-gateway.yml
        └─ application.yml
        └─ inventory-service.yml
        └─ notification-service.yml
        └─ notification-service-prod.yml
        └─ order-service.yml
        └─ product-service.yml
```

```
1 eureka:
2   client:
3     service-url:
4       defaultZone: http://localhost:8761/eureka/
5   fetch-registry: true
6   register-with-eureka: true
7
8 spring:
9   threads:
10     virtual:
11       enabled: true
```

```
1 eureka:
2   client:
3     service-url:
4       defaultZone: http://localhost:8761/eureka/
5
6 server:
7   port: 0
8
9 logging:
10   level:
11     root: INFO
12     com.ecommerce: INFO
13
14 # --- NUEVA CONFIGURACIÓN RETRY Y SPRING ---
15 spring:
16   threads:
17     virtual:
18       enabled: true
19   rabbitmq:
20     listener:
```

→ commitear:

Ejecución y Validación

Verificando el cambio

Para validar el cambio correctamente:

- 1 **Reinicia el API Gateway**
Observa que el arranque sigue siendo igual de rápido
- 2 **Preparado para picos de tráfico**
La JVM está ahora preparada para manejar picos que antes causaban "Too many threads"
- 3 **Notification Service desbloqueado**
El envío de correos vía SMTP no bloqueará el hilo que escucha RabbitMQ

✅ **Sin cambios visibles:** La belleza de los Virtual Threads es que son transparentes. El código no cambia, pero la capacidad de concurrencia se multiplica.

Virtual Threads: Resumen

- 🚪 **API Gateway:** Puerta de entrada con capacidad ilimitada de concurrencia
- ✉️ **Notification Service:** Envío de correos sin bloquear el consumo de eventos
- ⚡ **Una línea de config:** `spring.threads.virtual.enabled: true`
- 🔒 **Impacto:** Miles de peticiones simultáneas sin aumentar memoria
- 🔧 **Transparente:** El código no cambia, la escalabilidad sí

Con esto, tu Gateway y tu sistema de notificaciones están preparados para escalar sin límites de hilos del sistema operativo.

151. El problema de la dependencia de disponibilidad

➔ Si RabbitMQ no está disponible, entonces se implementa **patrón Transactional Outbox**: Implementación del mecanismo que desacopla el guardado del pedido de la entrega del mensaje, garantizando zero data loss.

** En la implementación de order, al estar acoplado con RabbitMQ, lanza un error 500 cuando el servicio de Rabbit está caído, tiene un acoplamiento fuerte.

https://micrometer.io/docs/tracing#_logging_context_propagation

152. Outbox: persistencia atómica 1

La Verdad Dolorosa

Si RabbitMQ cae, la tienda deja de vender

En la clase anterior vimos que cuando RabbitMQ se cae, el `@Transactional` hace rollback y la venta no se completa.

✗ Antes (Sistema Bloqueado)

- 1 Cliente intenta comprar
- 2 RabbitMQ está caído
- 3 Rollback → Venta rechazada

✓ Con Outbox (Resiliente)

- 1 Cliente intenta comprar
- 2 Mensaje guardado en DB
- 3 Venta completada ✓

Analogía:

Como un Email en tu Bandeja de Salida

Una metáfora que todos entendemos

La Bandeja de Salida

Cuando escribes un email sin internet, tu cliente de correo **no falla**. En su lugar, guarda el email en la "**Bandeja de Salida**" y lo envía automáticamente cuando recuperas conexión.



💡 **La Idea Central:** En lugar de intentar enviar el evento a RabbitMQ mientras el cliente espera, lo guardamos en una tabla dentro de PostgreSQL. Si la base de datos funciona, la venta se completa.

El Guardado Atómico

Todo en una sola transacción

Dentro de un mismo `@Transactional`, guardamos **dos cosas**:

Atomicidad Total

- 1 El Pedido en la tabla `t_orders`
- 2 El Evento en la tabla `outbox_events` (JSON del mensaje)

```
@Transactional
public OrderResponse placeOrder(OrderRequest request) {
    // 1. Guardar el pedido
    Order order = orderRepository.save(newOrder);

    // 2. Guardar el evento en Outbox (NO enviarlo a RabbitMQ)
    OutboxEvent event = new OutboxEvent(order, "PENDING");
    outboxRepository.save(event);

    // ✅ COMMIT - Ambas cosas están seguras
    return toResponse(order);
}
```

✅ **Garantía ACID:** Si PostgreSQL dice 'OK', ambas cosas se guardaron. Si falla, ninguna. RabbitMQ ya no participa en la transacción.

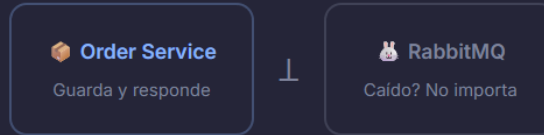
El Desacoplamiento Total

RabbitMQ ya no puede bloquear el negocio

El microservicio ya no necesita que RabbitMQ esté vivo para terminar su tarea con éxito.

🔒 Independencia Total

Ahora el flujo es completamente diferente:



✅ Beneficios Inmediatos

- 1 La venta se completa siempre (si la DB funciona)
- 2 Cliente recibe **201 Created** inmediatamente
- 3 El mensaje está "asegurado" en la base de datos

Project ▾



OutboxEvent.java ×

order-service

> .mvn

> src

> main

> java

> com.ecommerce.order_service

> config

> controller

> dto

> event

> exception

> listener

> mapper

> model

© Order

© OrderLineItems

ⓔ OrderStatus

© OutboxEvent

> repository

> service

⚙️ OrderServiceApplication

> resources

> test

> target

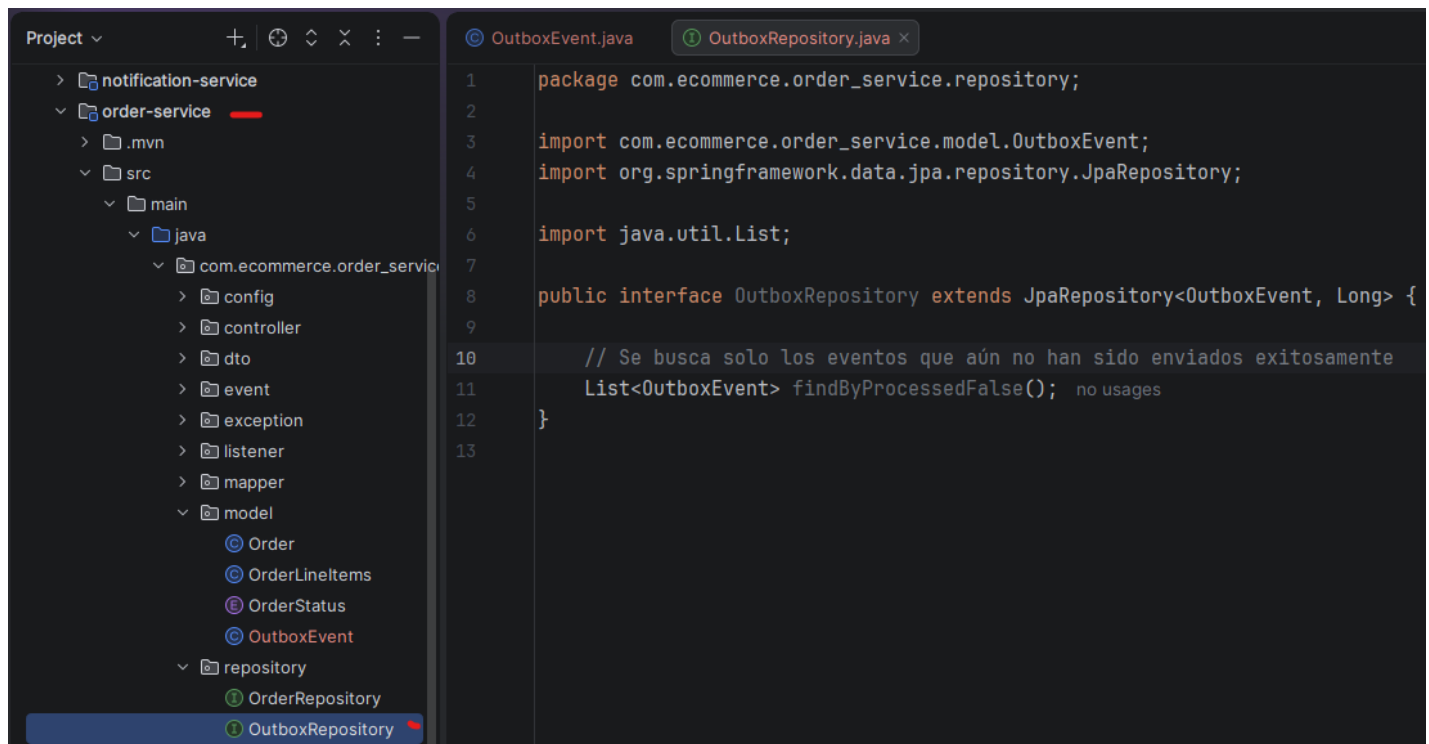
≡ .gitattributes

🔗 .gitignore

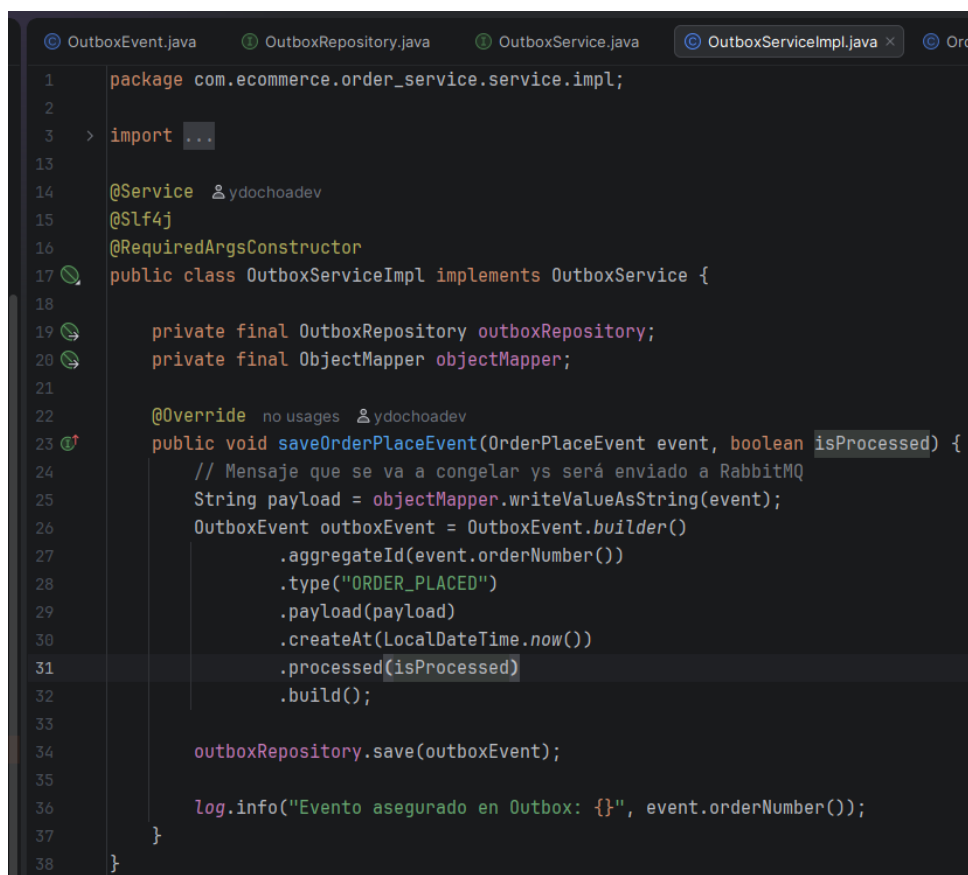
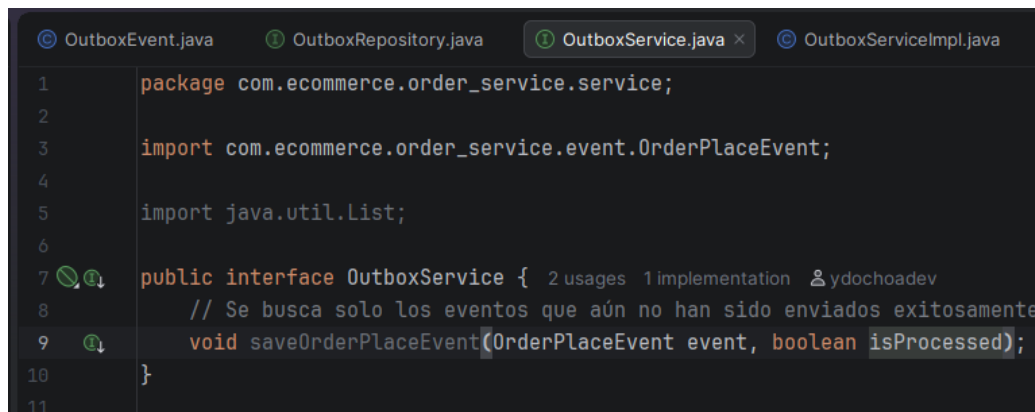
📖 HELP.md

📄 mvnw

```
1 package com.ecommerce.order_service.model;
2
3 > import ...
4
5
6
7
8
9
10
11
12
13
14
15
16 @Entity no usages
17 @Table(name = "outbox_events")
18 @Data
19 @AllArgsConstructor
20 @NoArgsConstructor
21 @Builder
22 public class OutboxEvent {
23
24     @Id
25     @GeneratedValue(strategy = GenerationType.IDENTITY)
26     private Long id;
27
28     private String aggregateId;
29
30     @Column(columnDefinition = "TEXT")
31     private String type;
32
33     private String payload;
34
35     private LocalDateTime createTime;
36
37     private boolean processed;
38 }
39
```



→ Se crea el servicio y la implementación en order-service:



153. Outbox: persistencia atómica 2

→ En la implementación de order:

```
© OrderServiceImpl.java x
1  package com.ecommerce.order_service.service.impl;
2
3  > import ...
24
25  @Service @ydochoadev *
26  @RequiredArgsConstructor
27  @Slf4j
28  @RefreshScope
29  public class OrderServiceImpl implements OrderService {
30      private final OrderRepository orderRepository;
31      private final OrderMapper orderMapper;
32      // private final WebClient.Builder webClientBuilder;
33      // private final InventoryClient inventoryClient; ya no s
34      private final RabbitTemplate rabbitTemplate;
35      private final OutboxService outboxService;
36
```

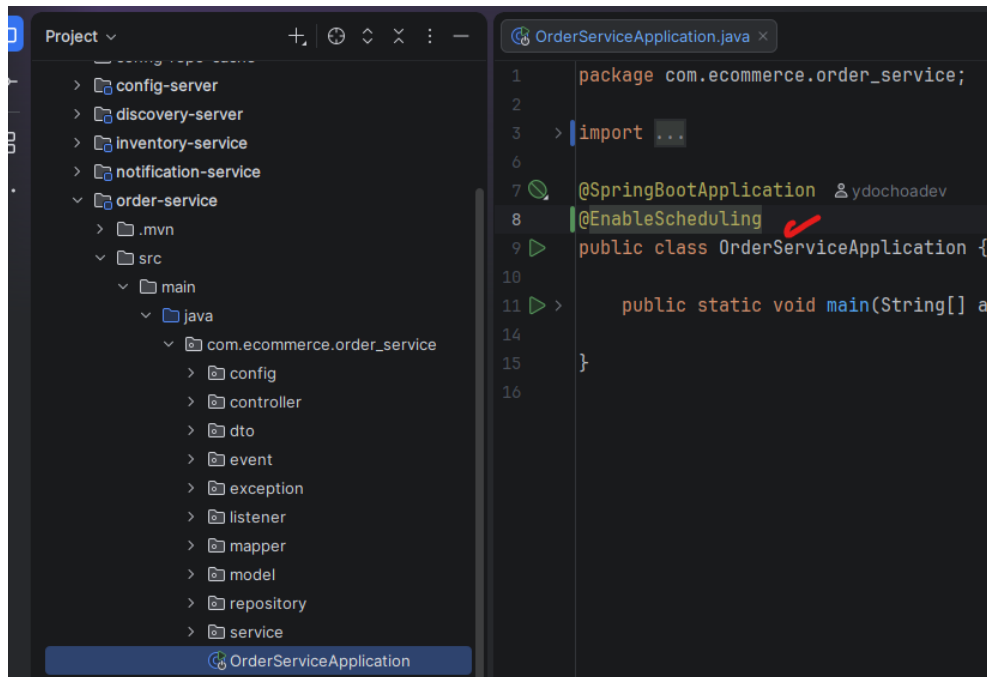
```
© OrderServiceImpl.java x
29  public class OrderServiceImpl implements OrderService {
46
47      @Override @ydochoadev *
48      @Transactional
49      /* @CircuitBreaker(name = "inventory", fallbackMethod = "fallbackMethod")
50      @Retry(name = "inventory") */
51      public OrderResponse placeOrder(OrderRequest orderRequest, String userId) {
52          if (!ordersEnabled) {
53              log.warn("Pedido rechazado: Servicio deshabilitado por configuración");
54              throw new RuntimeException("El servicio de pedidos está en mantenimiento");
55          }
56
57          log.info("Colocando nueva orden...");
58          Order order = orderMapper.toOrder(orderRequest);
59          order.setOrderNumber(UUID.randomUUID().toString());
60          // Estado del pedido
61          order.setStatus(OrderStatus.PLACED);
62          // Guardamos y capturamos la entidad persistida
63          Order savedOrder = orderRepository.save(order);
64          log.info("Orden guardada con éxito. ID: {}", savedOrder.getId());
65          // Contrato
66          List<OrderPlaceEvent.OrderItemEvent> orderItems = order.getOrderLineItemsList() List<OrderLineItems>
67              .stream() Stream<OrderLineItems>
68              .map( OrderLineItems item -> new OrderPlaceEvent.OrderItemEvent(
69                  item.getSku(), item.getPrice().toString(), item.getQuantity()
70              )) .toList();
71          // RabbitMQ
72          OrderPlaceEvent event = new OrderPlaceEvent(savedOrder.getOrderNumber(), orderRequest.getEmail(), orderItems);
73          boolean sendToRabbitMQ = false;
74          try {
75              rabbitTemplate.convertAndSend( exchange: "order-events", routingKey: "order.placed", event);
76              sendToRabbitMQ = true;
77              log.info("Mensaje enviado inmediatamente a RabbitMQ: {}", savedOrder.getOrderNumber());
78          } catch (AmqpException e) {
79              log.error("RabbitMQ caído. El outbox asegurará el envío posterior para la orden: {}", savedOrder.getOrderNumber());
80          }
81          // Outbox
82          outboxService.saveOrderPlaceEvent(event, sendToRabbitMQ);
83          log.info("Evento enviado a RabbitMQ para la orden: {}", savedOrder.getOrderNumber());

```


154. Message relay: El cartero de mensajes

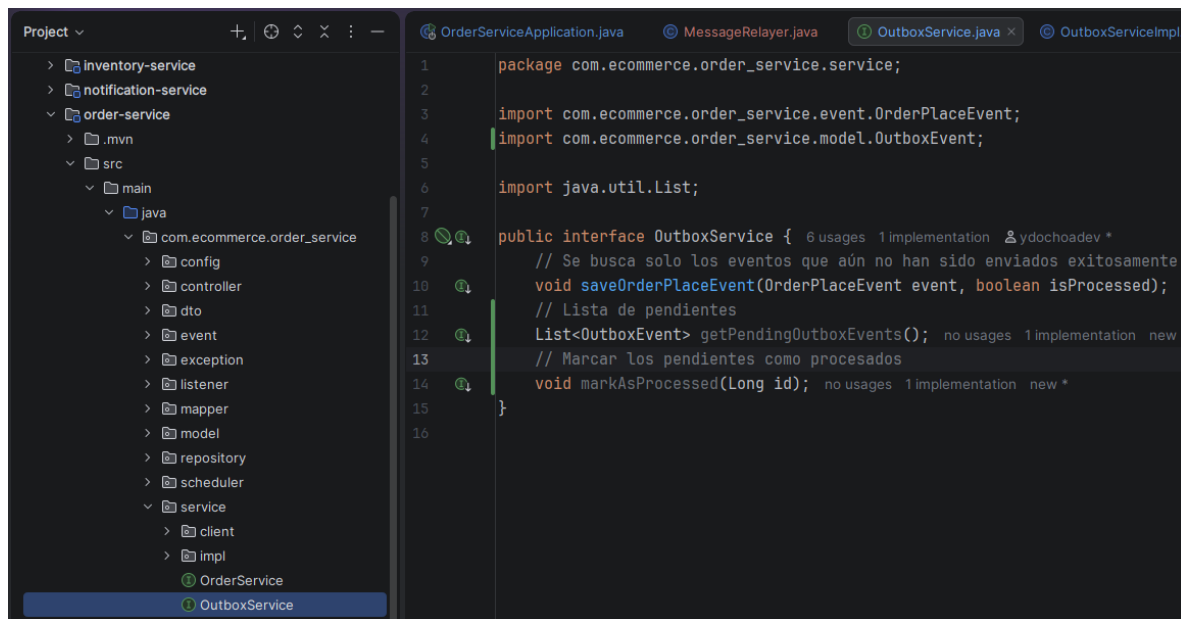
→ El pedido quedó en estado Placed, se debe crear un componente que trabajará como un cartero que cada 10 segundos revisa la bandeja de salida en Postgress, si encuentra un mensaje pendiente lo envía a RabbitMQ (Tarea programada).

→ En order-service: Habilitar las tareas programadas:

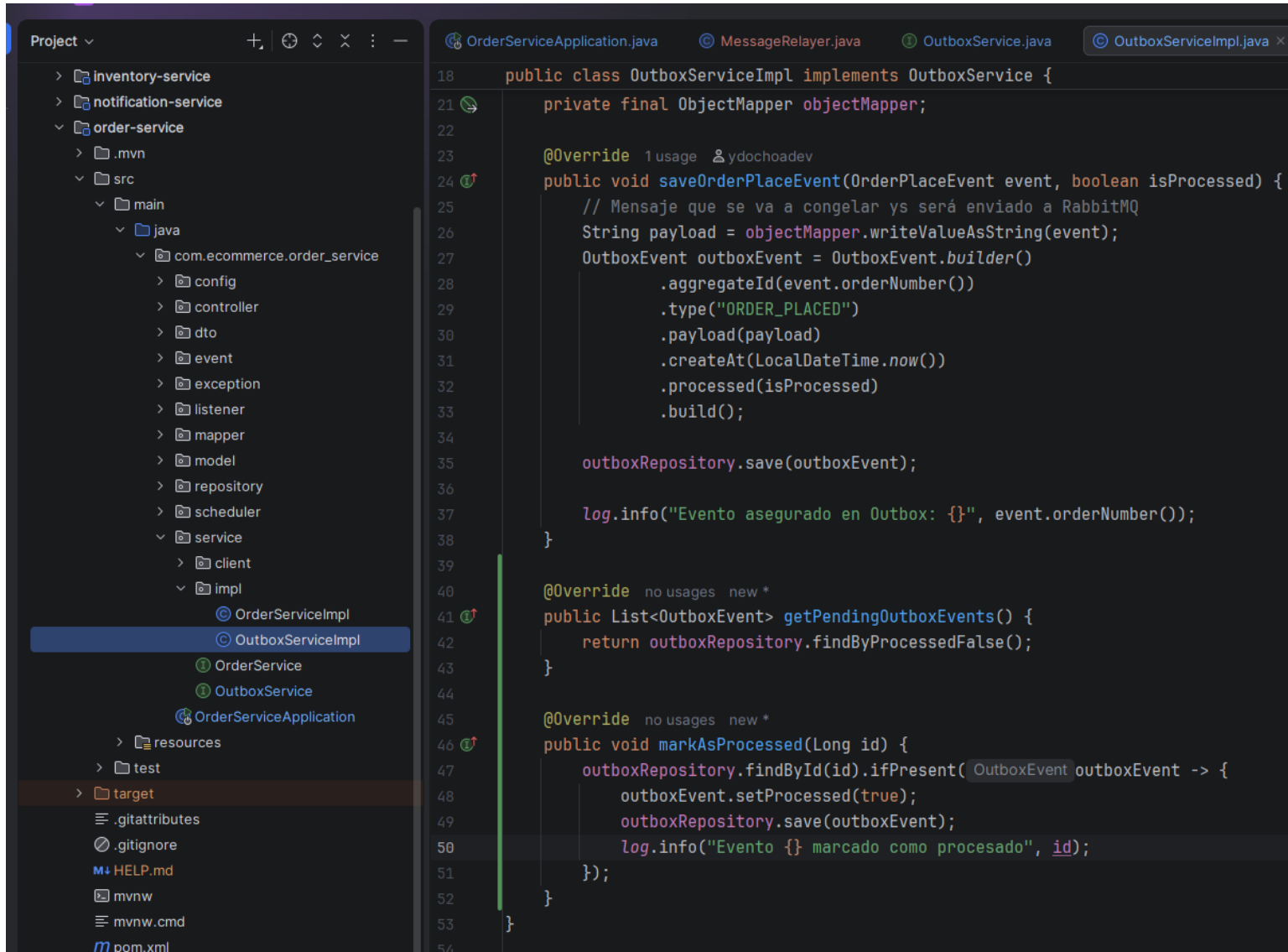


```
1 package com.ecommerce.order_service;
2
3 import ...
4
5
6
7 @SpringBootApplication
8 @EnableScheduling
9 public class OrderServiceApplication {
10
11     public static void main(String[] args) {
12
13     }
14
15 }
16
```

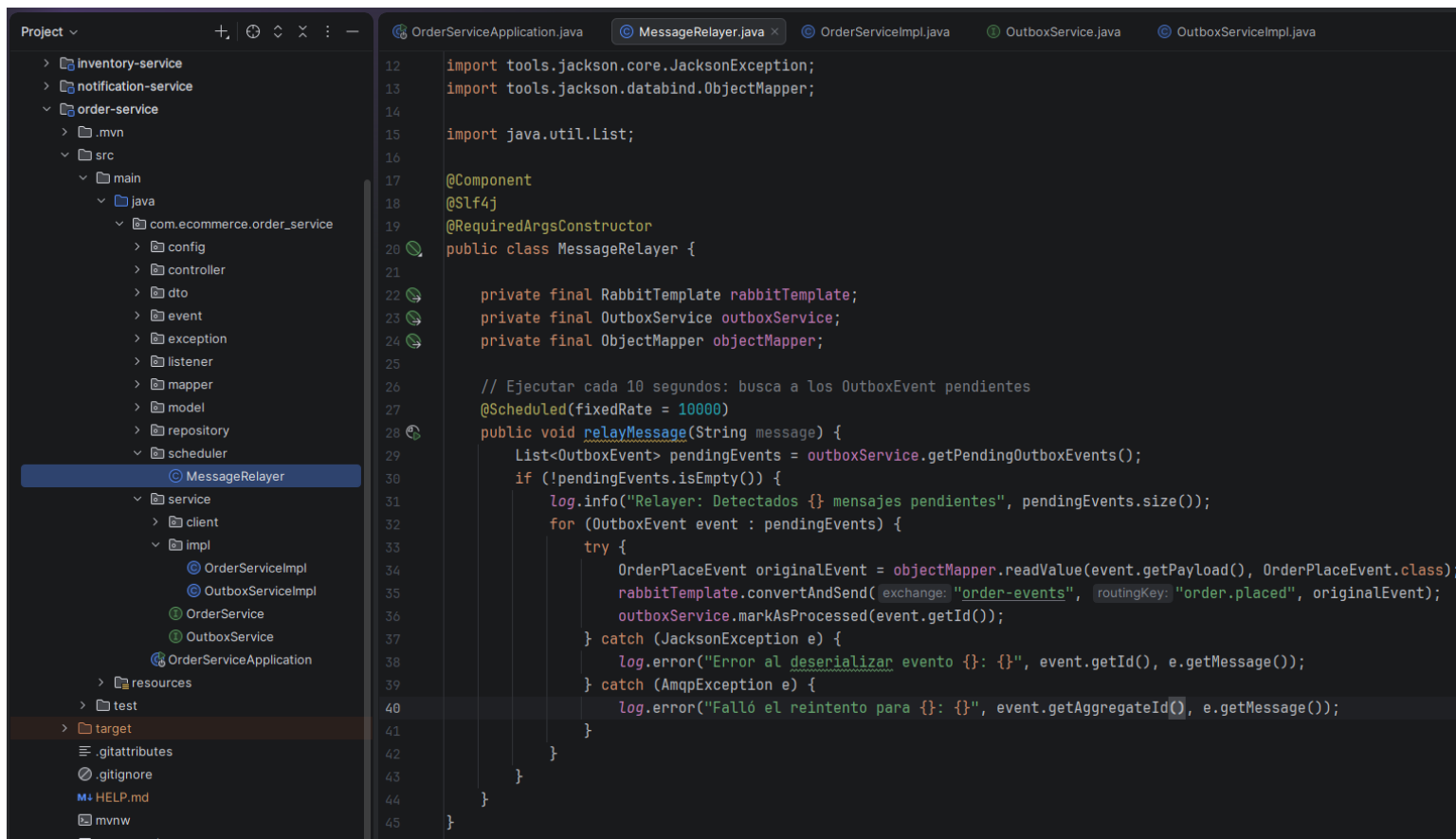
→ Agregar métodos para obtener los eventos no procesados y para actualizar su estado:



```
1 package com.ecommerce.order_service.service;
2
3 import com.ecommerce.order_service.event.OrderPlaceEvent;
4 import com.ecommerce.order_service.model.OutboxEvent;
5
6 import java.util.List;
7
8 public interface OutboxService {
9     // Se busca solo los eventos que aún no han sido enviados exitosamente
10     void saveOrderPlaceEvent(OrderPlaceEvent event, boolean isProcessed);
11     // Lista de pendientes
12     List<OutboxEvent> getPendingOutboxEvents();
13     // Marcar los pendientes como procesados
14     void markAsProcessed(Long id);
15 }
16
```

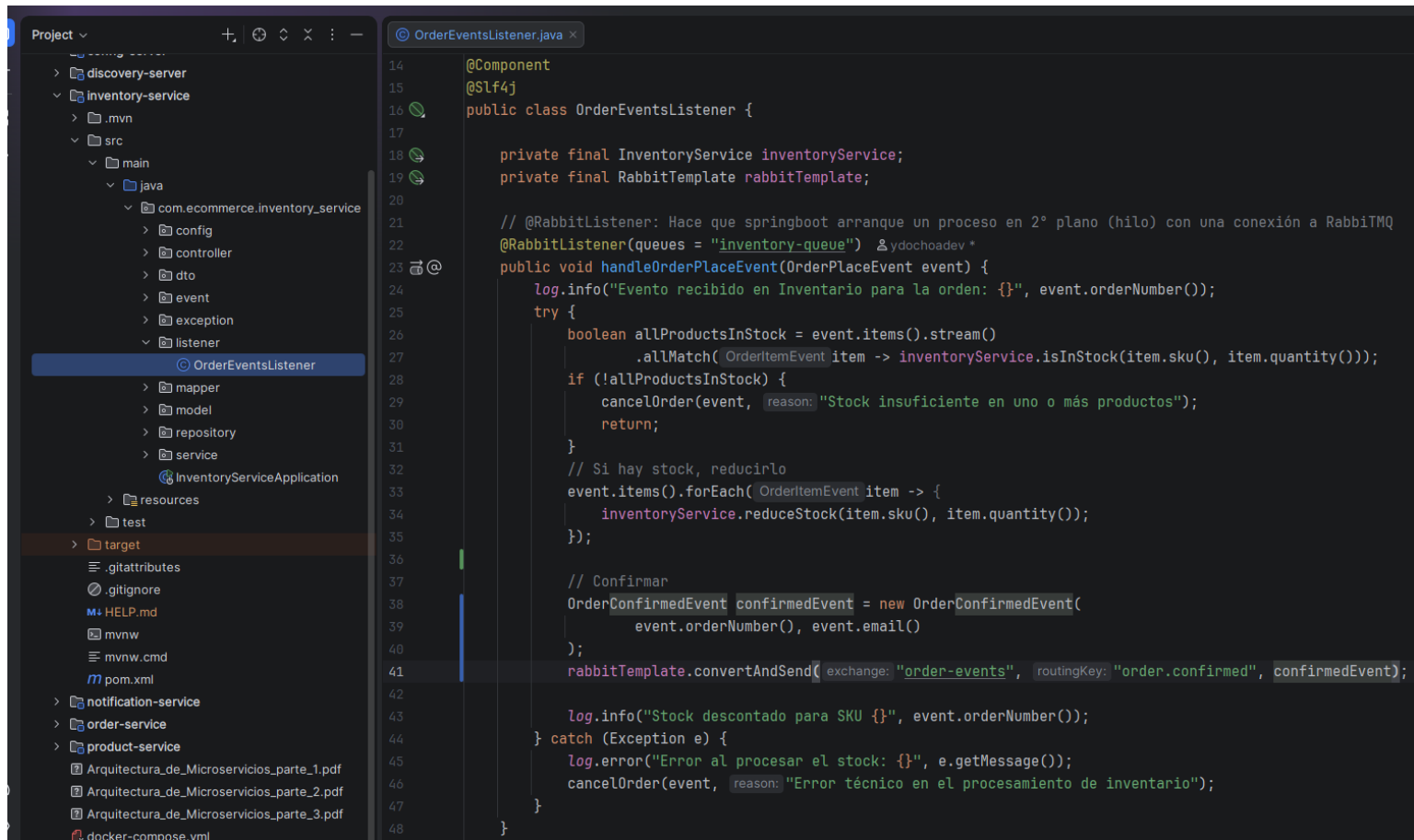



➔ Nueva clase para los eventos no procesados:



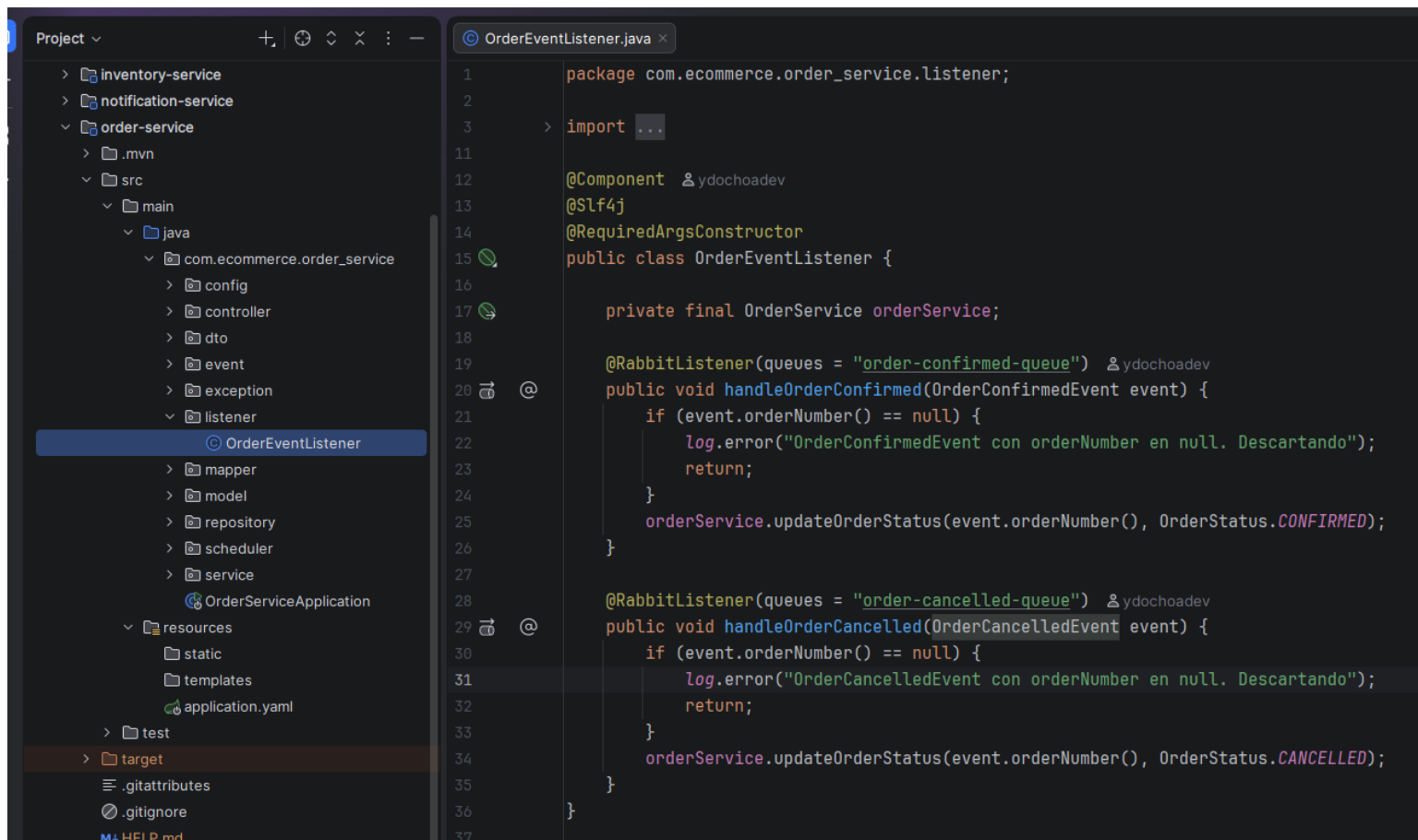
155. Transactional Outbox y Coreografía extrema

➔ Refactorizar el listener de inventory-service para que pueda descontar del inventario el producto seleccionado:



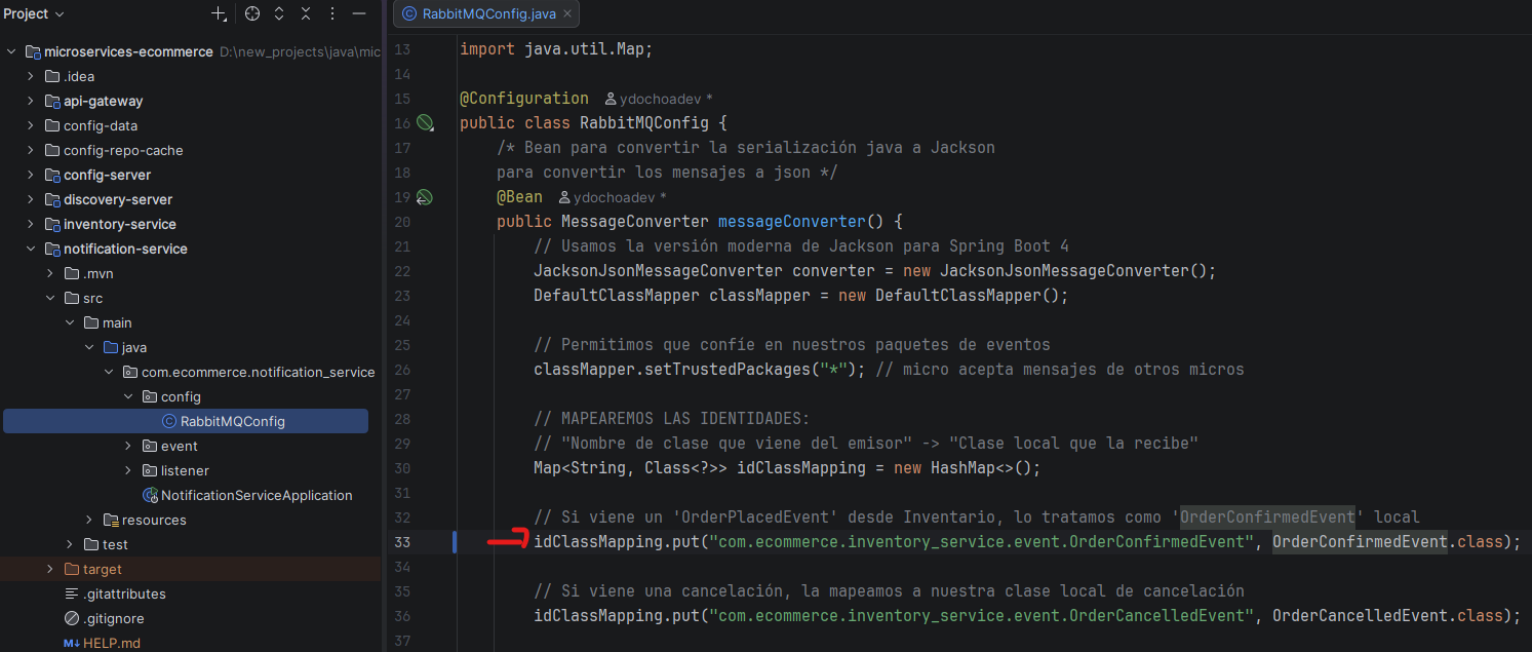
```
14 @Component
15 @Slf4j
16 public class OrderEventsListener {
17
18     private final InventoryService inventoryService;
19     private final RabbitTemplate rabbitTemplate;
20
21     // @RabbitListener: Hace que springboot arranque un proceso en 2º plano (hilo) con una conexión a RabbitMQ
22     @RabbitListener(queues = "inventory-queue") &ydochoadev *
23     public void handleOrderPlaceEvent(OrderPlaceEvent event) {
24         log.info("Evento recibido en Inventario para la orden: {}", event.orderNumber());
25         try {
26             boolean allProductsInStock = event.items().stream()
27                 .allMatch( OrderItemEvent item -> inventoryService.isInStock(item.sku(), item.quantity()));
28             if (!allProductsInStock) {
29                 cancelOrder(event, reason: "Stock insuficiente en uno o más productos");
30                 return;
31             }
32             // Si hay stock, reducirlo
33             event.items().forEach( OrderItemEvent item -> {
34                 inventoryService.reduceStock(item.sku(), item.quantity());
35             });
36
37             // Confirmar
38             OrderConfirmedEvent confirmedEvent = new OrderConfirmedEvent(
39                 event.orderNumber(), event.email()
40             );
41             rabbitTemplate.convertAndSend( exchange: "order-events", routingKey: "order.confirmed", confirmedEvent);
42
43             log.info("Stock descontado para SKU {}", event.orderNumber());
44         } catch (Exception e) {
45             log.error("Error al procesar el stock: {}", e.getMessage());
46             cancelOrder(event, reason: "Error técnico en el procesamiento de inventario");
47         }
48     }
```

➔ En order-service, listener:



```
1 package com.ecommerce.order_service.listener;
2
3 > import ...
4
11
12 @Component &ydochoadev
13 @Slf4j
14 @RequiredArgsConstructor
15 public class OrderEventListener {
16
17     private final OrderService orderService;
18
19     @RabbitListener(queues = "order-confirmed-queue") &ydochoadev
20     public void handleOrderConfirmed(OrderConfirmedEvent event) {
21         if (event.orderNumber() == null) {
22             log.error("OrderConfirmedEvent con orderNumber en null. Descartando");
23             return;
24         }
25         orderService.updateOrderStatus(event.orderNumber(), OrderStatus.CONFIRMED);
26     }
27
28     @RabbitListener(queues = "order-cancelled-queue") &ydochoadev
29     public void handleOrderCancelled(OrderCancelledEvent event) {
30         if (event.orderNumber() == null) {
31             log.error("OrderCancelledEvent con orderNumber en null. Descartando");
32             return;
33         }
34         orderService.updateOrderStatus(event.orderNumber(), OrderStatus.CANCELLED);
35     }
36 }
37
```

➔ En notification-service, configuración de RabbitMQ, debe escuchar a los eventos de confirmación o declinación:



```
13 import java.util.Map;
14
15 @Configuration & ydochoadev *
16 public class RabbitMQConfig {
17     /* Bean para convertir la serialización java a Jackson
18     para convertir los mensajes a json */
19     @Bean & ydochoadev *
20     public MessageConverter messageConverter() {
21         // Usamos la versión moderna de Jackson para Spring Boot 4
22         JacksonJsonMessageConverter converter = new JacksonJsonMessageConverter();
23         DefaultClassMapper classMapper = new DefaultClassMapper();
24
25         // Permitimos que confie en nuestros paquetes de eventos
26         classMapper.setTrustedPackages("*"); // micro acepta mensajes de otros micros
27
28         // MAPEAREMOS LAS IDENTIDADES:
29         // "Nombre de clase que viene del emisor" -> "Clase local que la recibe"
30         Map<String, Class<?>> idClassMapping = new HashMap<>();
31
32         // Si viene un 'OrderPlacedEvent' desde Inventario, lo tratamos como 'OrderConfirmedEvent' local
33         idClassMapping.put("com.ecommerce.inventory_service.event.OrderConfirmedEvent", OrderConfirmedEvent.class);
34
35         // Si viene una cancelación, la mapeamos a nuestra clase local de cancelación
36         idClassMapping.put("com.ecommerce.inventory_service.event.OrderCancelledEvent", OrderCancelledEvent.class);
37     }
```